MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING

MÓDULO 7. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO (LINUX) – INTRODUCCIÓN A LINUX

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

INDICE DE CONTENIDOS

1. Introducción 4
1.1. Distribución Linux 5
1.1.1. GNU: GNU's Not Unix! 6
1.1.2. Kernel de Linux 8
1.1.3. Componentes de la GUI (Interfaz gráfica de usuario) 9
1.1.4. Drivers 9
1.1.5. Gestores de paquetes 10
2. Sistema de permisos 11
2.1. Estructura de directorios Linux 11
2.2. Sistema de usuarios 13
2.2.1. Administración de usuarios en Linux 13
2.2.2. Sistema de permisos de ficheros 16
3. Fuentes del software 19
3.1. Paquetes, gestores de paquetes y repositorios 20
3.2. Estructura de un paquete Debian 20
3.3. ¿Qué hace este paquete debian? 22
3.4. Gestores de paquetes (APT) 22
3.5. Gestión de repositorios software (sources) 23
3.6. Gestión de vulnerabilidades conocidas 24
3.7. Gestión de paquetes en otras distribuciones Linux. 26
3.7.1. Gestor de paquetes ARP. 26
3.7.2. Diferencias entre paquetes ARP y DEBIAN 27
3.7.3. Otros formatos de paquetes usados en Linux 27
4. Gestión y limitación del uso de recursos 27
4.1. El directorio /etc/security 28
4.2. Estableciendo límites de uso de recursos 30
4.3. Lista de recursos a limitar en /etc/security/limits.conf 31
4.4. Acotando el uso que un proceso hace de la CPU 32

4.5. Cuotas de disco 33

5. Gestión de redes en sistemas Linux 35

5.1 Servicios mínimos 35

5.1.1 NTP 35

5.1.2 DNS 36

5.2 Firewall 39

5.2.1 Comandos básicos 39

5.2.2 Filtrado de tráfico 40

5.3 Traffic control 40

5.3.1 QDISC 41

5.3.2 Clases 41

5.3.3 Filtros 41

5.3.4 QDISC sin clases 41

5.3.5 QDISC con clases 42

5.3.6 tc 42

5.4 inet/xinet 43

5.5 Componentes de red e interfaces 44

5.5.1 Configurar una interfaz de red 44

5.5.2 Crear una interfaz de red virtual 45

5.5.3 Tabla de enrutamiento de IPs 45

5.6 Herramientas útiles 45

BIBLIOGRAFÍA 47

1. INTRODUCCIÓN

Un poco de historia …
https://es.wikipedia.org/wiki/Historia_de_Linux
https://es.wikipedia.org/wiki/N%C3%BAcleo_Linux

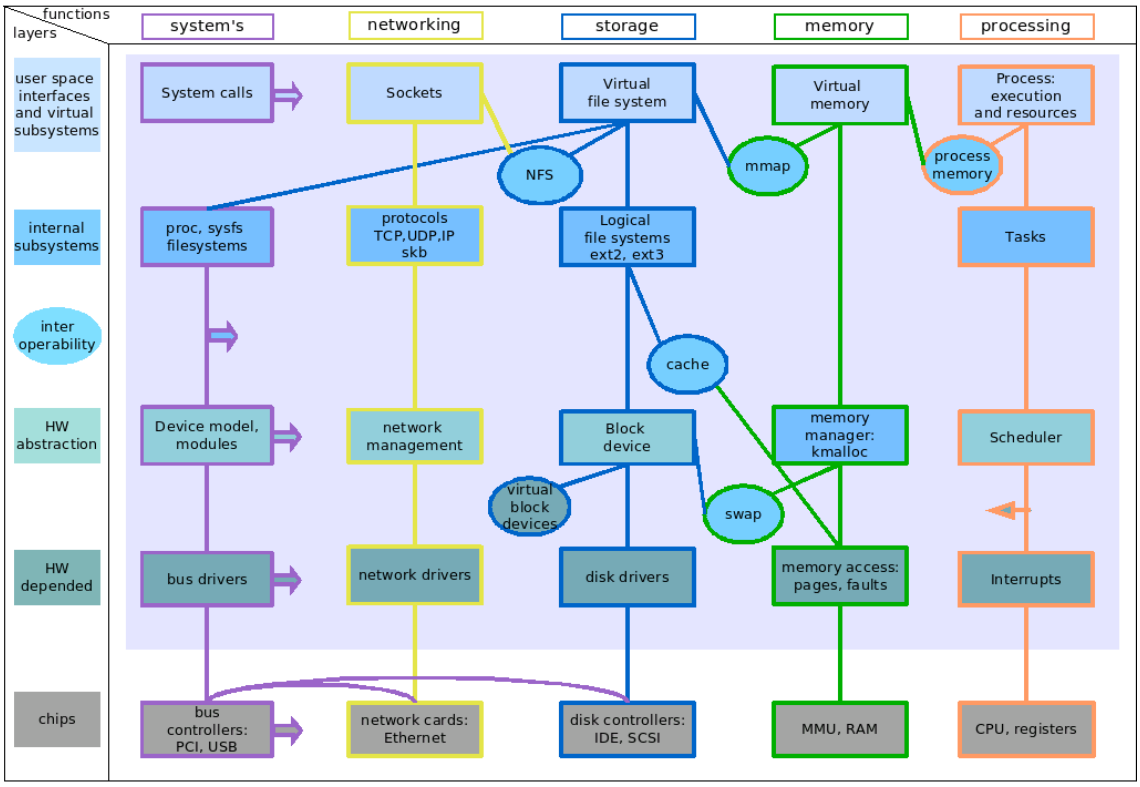
Estadísticas de uso …
https://en.wikipedia.org/wiki/Usage_share_of_operating_systems

Lista de distribuciones …
https://en.wikipedia.org/wiki/List_of_Linux_distributions

Repositorio del kernel …
https://github.com/torvalds/linux

Just another kernel map … (Mapa interactivo del kernel)
http://www.makelinux.net/kernel_map/

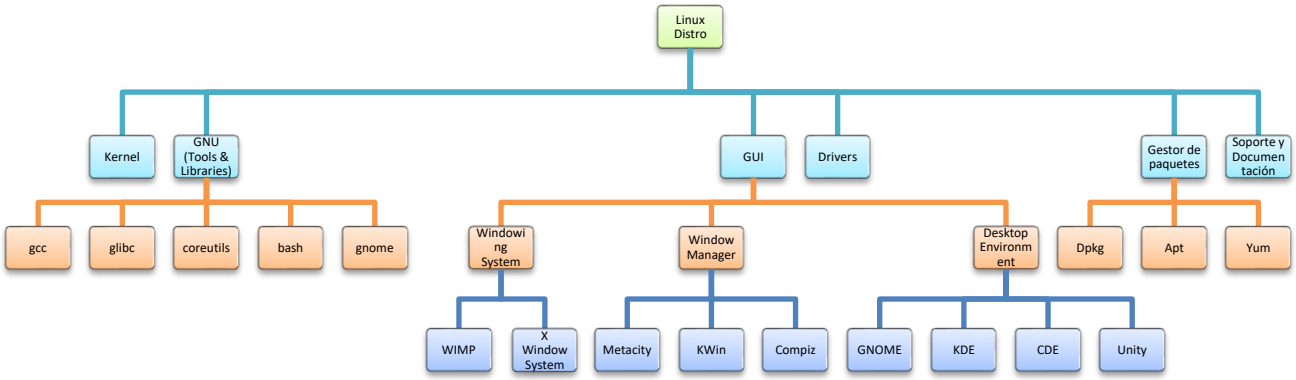
Simplified Linux kernel diagram in form of a matrix map



Designed with OpenOffice.org by (cc) (by-nc-sa) Constantine Shulyupin, www.linuxdriver.co.il

1.1. DISTRIBUCIÓN LINUX

Una distribución Linux se compone principalmente de 6 elementos, el **kernel** de Linux, una serie de herramientas y librerías llamadas **GNU**, los sistemas que se encargan de la **interfaz gráfica**, **drivers**, un sistema **gestor de paquetes** que nos permite instalar nuevas aplicaciones y una plataforma de documentación junto con una **comunidad** de desarrollo. Todos estos componentes juntos es lo que se conoce como una distribución Linux o, con matices, un sistema operativo Linux. Ejemplos de distribuciones Linux son Ubuntu, Centos, Red Hat, Debian, …



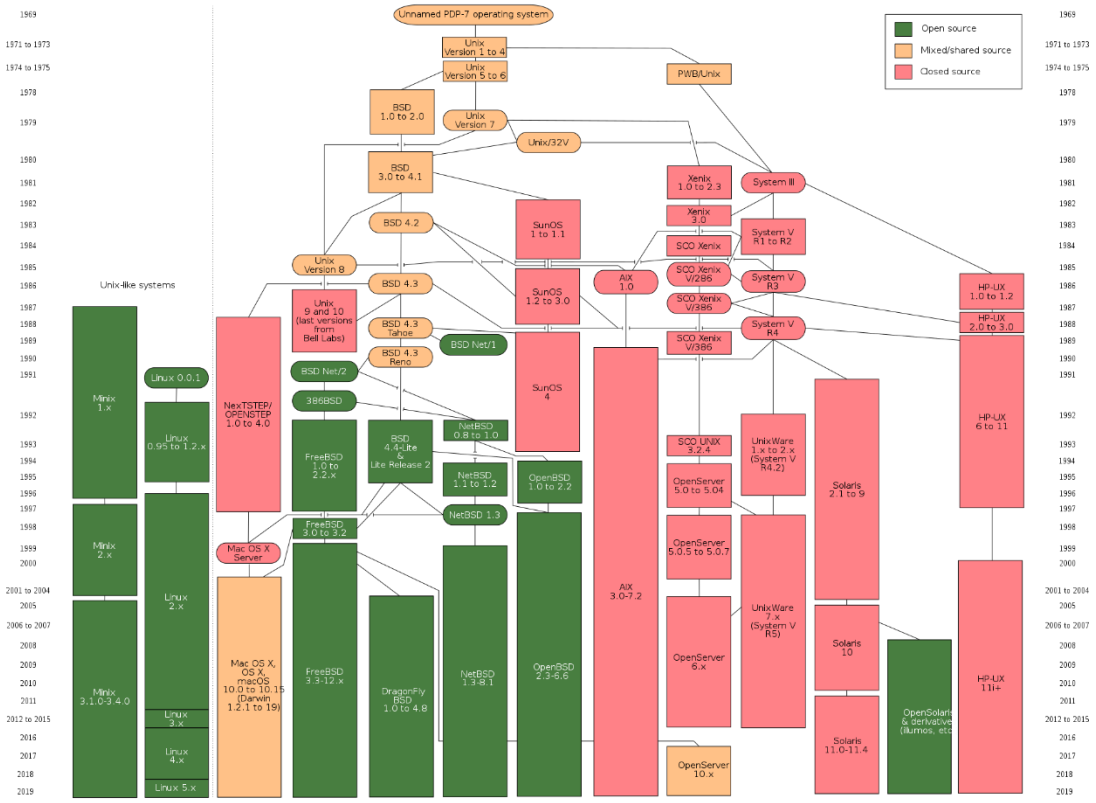
Algunas de las distribuciones Linux existentes …

1.1.1. GNU: GNU's Not Unix!

GNU es un sistema operativo creado por Richard Stallman junto con un conjunto de herramientas software adicionales. **El componente GNU que se incluye en una distribución Linux se refiere a las herramientas adicionales**.

Antes de nada, aclaremos algunos conceptos:

- **UNIX®**: Es un sistema operativo propietario desarrollado en 1969 por un grupo de empleados de los laboratorios Bell de AT&T, entre los que figuran Dennis Ritchie, Ken Thompson y Douglas McIlroy. UNIX ha servido como base o inspiración de muchos otros sistemas operativos



Familias UNIX más significativas

- AT&T: La familia que tuvo su origen en el UNIX de AT&T. Considerada la familia UNIX "pura" y original.
- BSD: Familia originada por el licenciamiento de UNIX a Berkely. BSD se reescribió para no incorporar propiedad intelectual originaria de AT&T en la versión 4. La primera

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

implementación de los protocolos TCP/IP que dieron origen a Internet son la pila (stack) TCP/IP BSD.

- AIX: Esta familia surge por el licenciamiento de UNIX System III a IBM.
- Xenix: Familia derivada de la adquisición de los derechos originales de AT&T primero por parte de Microsoft y de esta los vendió a SCO.
- GNU: En 1983, Richard Stallman anunció el Proyecto GNU, un ambicioso esfuerzo para crear un sistema similar a Unix, que pudiese ser distribuido libremente. El software desarrollado por este proyecto -por ejemplo, GNU Emacs y GCC - también han sido parte fundamental de otros sistemas UNIX.
- Linux: En 1991, cuando Linus Torvalds empezó a proponer el *núcleo* Linux y a reunir colaboradores, las herramientas GNU eran la elección perfecta. Al combinarse ambos elementos, conformaron la base del sistema operativo (basado en POSIX), que hoy se conoce como GNU/Linux.

- **GNU/Linux:** Es un conjunto de sistemas operativos libres multiplataforma, multiusuario y multitarea basados en UNIX. El sistema es la combinación de varios proyectos, entre los cuales destacan GNU, encabezado por Richard Stallman y la Free Software Foundation junto con el núcleo o kernel «Linux», programado por Linus Torvalds. Su desarrollo es uno de los ejemplos más prominentes de software libre: todo su código fuente puede ser utilizado, modificado y redistribuido libremente por cualquiera, bajo los términos de la licencia GPL —**L**icencia **P**ública **G**eneral de GNU— y otra serie de licencias libres.[1]

¿qué herramientas componen GNU?

- **GNU Compiler Collection**: Conjunto de compiladores estándar en los sistemas operativos derivados de UNIX.
- **Glibc**: Librería estándar para el lenguaje de programación C. Actualmente también soporta C++ de forma directa e indirectamente muchos otros lenguajes.
- **Coreutils**: Implementación de herramientas útiles comunes en los sistemas basados en UNIX.

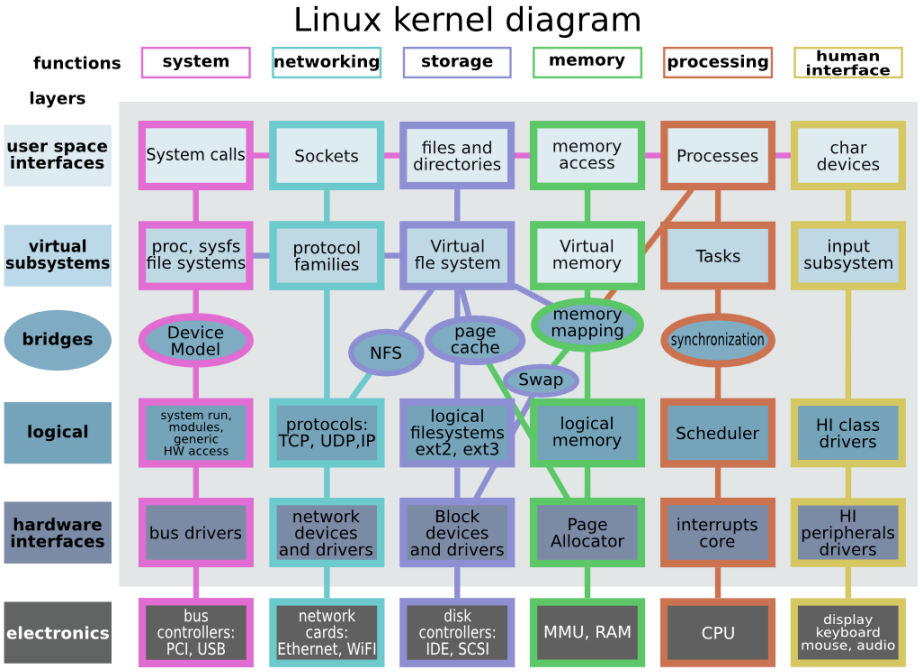
  Ejemplo:
  - **Cat**: Muestra el contenido de un fichero.
  - **Ls**: Lista el contenido de un directorio.
  - **Rm**: Borra el directorio o fichero indicado.
  - **Binutils**: Utilidades para crear y administrar programas binarios.

    Ejemplo:
    - Ficheros objeto

    - Librerias
    - Ensambladores
    - **Bash**: Shell Unix junto con el lenguaje de comandos. Es una de las shell mas usadas en las distribuciones Linux.
- **GNOME**: Entorno de escritorio Gnome. Implementa la metáfora de ser un escritorio de trabajo. Incorpora los elementos gráficos tales como gestión de ventanas, iconos, menus, archivos, fondos, ...

1.1.2. Kernel de Linux

El kernel de Linux fue inicialmente creado por Linux Torvalds en 1991 como un sistema operativo basado en Unix. El siguiente diagrama muestra una estructura de los diferentes elementos que nos podemos encontrar, también es posible consultar la estructura del kernel en este mapa interactivo. En combinación con las herramientas de GNU se conformó el sistema operativo GNU/Linux, que utiliza Linux como kernel. Esta parte del SO (al igual que en la mayoría), es relativamente pequeña, sin embargo es la más utilizada, por lo que suele residir en memoria de forma continua.



Los componentes del núcleo de Linux pueden ser clasificados a alto nivel según la función que desempeñan dentro del mismo:

- **Interfaz humana**: En él nos encontramos los drivers y dispositivos de interacción con el usuario (ratón, teclado, altavoces, etc.),
- **Procesamiento**: Componentes relacionados con la gestión de procesos, tareas, sincronización, planificación de procesos e interrupciones.

- **Memoria**: Se encargan de la virtualización, direccionamiento, intercambio de memoria a disco, asignación de páginas, etc.
- **Almacenamiento**: Interacción con los dispositivos de almacenamiento, gestión de sistemas de ficheros, implementación de sistemas de ficheros virtuales, cacheado de páginas, etc.
- **Redes**: Gestión de conexiones, familias de protocolos, interacción con los dispositivos físicos, etc.
- **Sistema**: Se encargan de las llamadas al sistema, los sistemas de ficheros virtuales de diagnostico del kernel, el modelo de dispositivo, etc.

1.1.3. Componentes de la GUI (Interfaz gráfica de usuario)

La interfaz gráfica de una distribución Linux es la capa que recoge la interacción visual con el usuario, es decir, facilita la interacción usuario-máquina, encargándose de proporcionar elementos fáciles e intuitivos de usar. Esta capa de un sistema Linux se divide en tres componentes, aunque algunas variantes suelen fusionar algunos de ellos: Sistema de ventanas, Administración de ventanas y Entorno de escritorio.

- **Sistema de ventanas**:
  Controla de forma separada diferentes partes de las ventanas de navegación, implementa los conceptos relacionados con el paradigma WIMP (Ventanas, Iconos, Menús, Punteros). Se encarga de la funcionalidad de esos conceptos dejando de lado aspectos tales como la apariencia.
- **Administrador de ventanas**:
  Gestiona el posicionamiento y la apariencia de las ventanas dentro del sistema de ventanas. Trabaja en conjunto con capas inferiores para acceder a recursos tales como tarjetas gráficas, ratón, teclado u otros dispositivos relacionados con el manejo de interfaz.
- **Entorno de escritorio**:
  Es el encargado de implementar la metáfora de escritorio. El entorno de escritorio suele ser usado sólo en ordenadores personales, siendo frecuente desactivar esta función en servidores. El entorno de escritorio normalmente se compone de iconos, ventanas, barras de herramientas, widgets, etc. Dentro de este componente se implementan habilidades típicas como arrastrar, diferentes funciones del ratón, atajos de teclado para el manejo de los elementos del escritorio, etc.

1.1.4. Drivers

Programa que se encarga de **controlar** un determinado **dispositivo hardware** adjunto en un ordenador, normalmente **proporciona una interfaz software** para poder interactuar de forma segura con el componente hardware que encapsula. De esta forma, cualquier programa que

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

desee interactuar con un dispositivo hardware, **no necesita conocer el funcionamiento** a bajo nivel del mismo, le basta con conocer la especificación software del driver a utilizar. Existe una variante de driver que no interactúan con un dispositivo físico, sino que lo hacen con **dispositivos virtuales** que emulan ser un dispositivo hardware. Véase discos duros virtuales, tarjetas de red, etc.

Existen una gran variedad de drivers pero todos ellos tienen algunos **atributos comunes**:

- **¡¡Los drivers corren como parte del kernel!!**, lo que implica una gran responsabilidad, fallos en un driver pueden dañar seriamente el sistema.
- Los drivers deben proporcionar una **interfaz estándar de comunicación** con el Kernel o con el subsistema al que pertenecen.
- **Usan servicios estándar del kernel** como asignación de memoria, entrega de interrupciones, colas de espera, etcl para su funcionamiento.
- **Permiten ser cargados bajo demanda** en el kernel según las necesidades. Ejemplo: al insertar un USB, el driver es cargado dinámicamente en el kernel.

Es posible **seleccionar** que **drivers** incorporar **cuando se está compilando el kernel**.

1.1.5. Gestores de paquetes

Conjunto de herramientas software que permiten automatizar el proceso de instalación, actualización, configuración y borrado de programas en un sistema operativo. En las distribuciones Linux, es frecuente encontrar diferentes opciones de gestores de paquetes.

Ejemplos de gestores de paquetes son: aptitude, apt, dpkg, pacman, homebrew, etc.

Funciones típicas de un gestor de paquetes:

- **Gestionar los inventarios** de software disponibles, así como los diferentes repositorios o fuentes desde los cuales es posible descargar nuevos paquetes.
- Trabajar con el sistema de ficheros para **extraer el contenido de los paquetes**, así como proporcionar la información anexa del mismo (nombre, descripción, desarrolladores, dependencias, etc).
- Facilitar al usuario la **localización de los paquetes** deseados **y la navegación** dentro del inventario.
- **Resolver las dependencias** necesarias durante el proceso de instalación de un paquete. Es decir, cuando un paquete se está instalando, en caso de que este necesite para su funcionamiento la instalación previa de otros paquetes, el gestor de paquetes debe ser capaz de instalarlos **en el correcto orden** para finalmente proceder a la instalación del paquete deseado.
- **Asegurar la integridad y autenticidad** de los paquetes instalados, así como de los que se van a instalar. Para realizar esta tarea, el gestor de paquetes verifica los certificados digitales de los repositorios de paquetes, así como las firmas digitales de los mismos para garantizar que las fuentes son correctas y que los paquetes no han sido modificados.

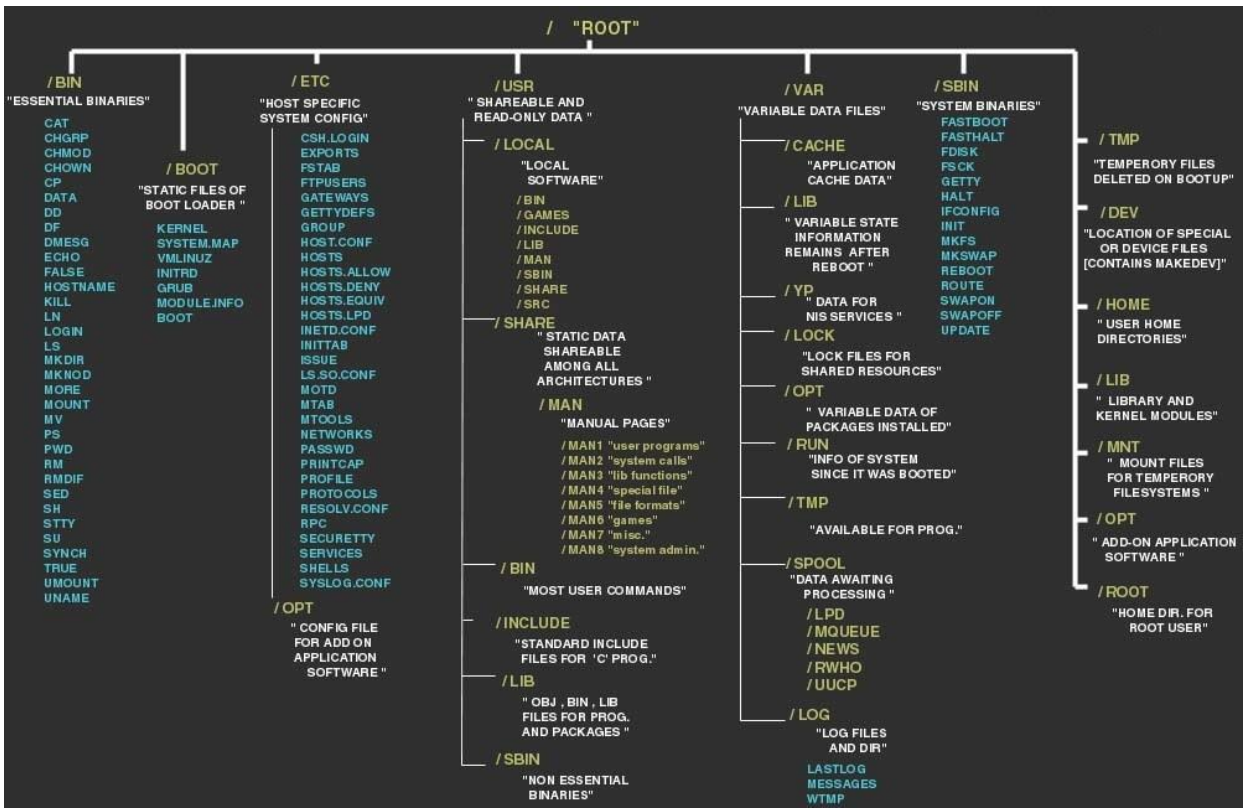
Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

2. SISTEMA DE PERMISOS

El sistema de permisos de Linux permite decidir para cada archivo en el sistema los usuarios permitidos y como pueden acceder.

2.1. ESTRUCTURA DE DIRECTORIOS LINUX

Dentro de una instalación Linux, disponemos de una estructura de directorios existentes los cuales conforman la propia instalación. La estructura principal de sistema de ficheros para los sistemas Linux y otros sistemas base-UNIX viene definida por la Filesystem Hierarchy Standard (FHS). Sin embargo, Linux contiene algunos directorios que aún no se encuentran en esta especificación.



En primer lugar, tenemos el directorio raíz, representado por la /. De él nace toda la jerarquía de directorios existente en el sistema. En Linux no se nombran los volúmenes con una notación específica como por ejemplo se haría en Windows con la existencia de unidades como C:\. Bajo el directorio raíz, encontramos los siguientes directorios:

- **/bin (binarios esenciales de usuario)**: ejecutables básicos para ser utilizados en la consola. Ejemplo: nano (editor de ficheros por consola).
- **/boot (ficheros estáticos de arranque)**: ficheros relacionados con el sistema de arranque del sistema operativo. Ejemplo: grub.

- **/dev**: por cada dispositivo conectado al sistema, existe un fichero en este directorio conteniendo información sobre su estado, así como interfaces que permiten la interacción con el mismo.
- **/etc (ficheros de configuración)**: contiene los ficheros de configuración de las aplicaciones que están instaladas en la instancia.
- **/home (directorios home)**: directorio de trabajo de los usuarios.
- **/lib (librerías esenciales compartidas)**: conjunto de librerías básicas compartidas, así como de algunos módulos del kernel.
- **/media (medios extraíbles)**: directorio para el montaje de sistemas de archivos de dispositivos extraíbles (CD-ROM, floppy, etc.)
- **/mnt (puntos de montaje temporales)**: directorio para el montaje de sistemas de fichero temporales.
- **/opt (paquetes opcionales)**: directorio para almacenar los paquetes software de las aplicaciones.
- **/sbin (binarios de administració)**: ejecutables del sistema. En el podemos encontrar utilidades como iptables, halt, fdisk, ...
- **/srv (datos de servicios)**: contiene datos para los servicios ofrecidos por el sistema (http, ftp, etc.)
- **/tmp (temporales)**: contiene ficheros temporales del sistema.
- **/user (binarios de usuario y datos read-only)**: contiene directorios destinados a utilidades de usuarios, así como las aplicaciones.
- **/var (contenido variable)**: contiene ficheros de contenido variable. Ejemplo: **logs**, directorios de correo, etc. Los datos que deberían ser escritos en /usr (solo lectura) se escriben aquí durante el modo normal de operación.
- **/root**: directorio de trabajo para el usuario root.
- **/proc (ficheros de kernel y procesos)**: sistema de ficheros virtual con información sobre el estado del kernel así como de los procesos en ejecución en la máquina.

Dependiendo de la distribución de Linux, pueden aparecer algunos directorios adicionales o incluso, ya obsoletos. Ejemplos:

- **/lost+found (ficheros recuperados):** Este directorio suele encontrarse en todas las distribuciones Linux. Cuando el sistema colapsa y es reiniciado, se ejecuta una comprobación del sistema de ficheros, almacenando aquí los ficheros corruptos encontrados, dando la posibilidad al usuario de recuperarlos en la medida de lo posible usando sus propios medios.
- /selinux (SELinux Virtual File System): En las distribuciones que usan el módulo del kernel SELinux para gestionar aspectos de seguridad como el control de acceso. Distribuciones como RedHat o Fedora usan este módulo. Es similar a /proc. Ubuntu no utiliza SELinux, por lo que la presencia de este directorio parece tratarse de un bug.

2.2. SISTEMA DE USUARIOS

Dentro de un entorno Linux, se entiende por **usuario** a una entidad bajo la cual van a poder ser ejecutados procesos en su nombre, los cuales dispondrán de los permisos de acceso asociados al usuario.

Para facilitar la gestión de permisos, los usuarios pueden pertenecer a **grupos**. Un usuario puede pertenecer a múltiples grupos, los cuales a su vez están compuestos por múltiples usuarios. Dentro del sistema de permisos de Linux es posible indicar por ejemplo que un fichero puede ser modificado por los procesos de los usuarios que pertenecen a un determinado grupo.

2.2.1. Administración de usuarios en Linux

Dentro de cualquier distribución Linux, nos encontramos utilidades que nos permiten facilitar la administración de usuarios. Con administración de usuarios nos referimos a realizar sobre estas entidades operaciones de creación, modificación y borrado. Los usuarios, disponen de una serie de parámetros de configuración típicos que cada uno puede personalizar según sus preferencias o las del administrador. Entre esas preferencias encontramos el directorio personal del usuario (home directory), las variables de entorno, la contraseña de acceso, la shell a utilizar, etc.

Veamos algunos comandos útiles:

- **useradd**: utilidad del sistema que permite la creación de usuarios.
- **adduser**: utilidad más "user friendly" e interactiva para la creación de usuarios. Internamente utiliza useradd.
- **userdel**: borra usuarios del sistema.
- **groupadd**: permite la creación de grupos.
- **groupdel**: borra grupos del sistema.
- **usermod**: modifica propiedades de la cuenta de un usuario.
- **passwd**: permite modificar la password de un usuario.
- **chsh**: permite cambiar la shell por defecto de un usuario. Según ciertas condiciones, es posible establecer shell que permiten restringir los comandos que se pueden lanzar en ellas, incluso para restringir el acceso a los usuarios al sistema.

El súper usuario. (root):

Cuenta de usuario especial usada para la administración del sistema. En la mayoría de sistemas Linux el nombre de usuario es root y su identificador de usuario (UID) es 0. La principal característica de esta cuenta es que puede realizar modificaciones a lo largo de todo el sistema de forma no restringida, pudiendo incluso llegar a dañarlo.

El comando sudo.

Permite ejecutar comandos con el usuario indicado. En caso de no indicarle ningún usuario, el comando se ejecuta con el usuario root.

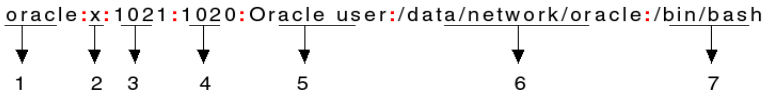
Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Así por ejemplo, mediante el comando sudo -su bob estaríamos ejecutando una shell (opción -s) con el usuario (opción -u) bob. Podríamos lanzar cualquier otro comando como por ejemplo ls: sudo -u bob ls.

¿Dónde se almacena toda esta configuración en el sistema?

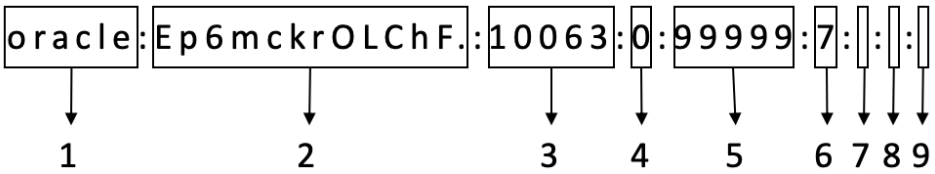
Para gestionar todo esto, los sistemas Linux hacen uso de una serie de ficheros de propiedades los cuales son modificados mediante la utilización de comandos de administración como los anteriores (aunque nada impide que se modifiquen directamente los ficheros, surtiendo el mismo efecto). Veamos los principales ficheros implicados:

- **/etc/passwd**: recoge información sobre la cuenta del usuario. Cada línea del fichero representa una cuenta de usuario, cada atributo se encuentra delimitado por ":" . En la imagen se trata del usuario oracle y tiene el siguiente formato.



5. nombre de usuario.
6. password del usuario. El valor x indica que el hash de la password ha sido almacenado en el fichero /etc/shadow.
7. Identificador numérico de usuario (UID). El usuario root, usa el UID=0
8. Identificador numérico de grupo primario (GUID).
9. Información descriptiva del usuario.
10. Path al directorio home del usuario.
11. Shell por defecto para el usuario.

- **/etc/shadow**: recoge el hash de la password del usuario, así como los valores de expiración de la cuenta. El fichero sigue el mismo patrón que el fichero passwd, es decir, una línea por usuario y cada atributo se encuentra delimitado por el carácter ":". De izquierda a derecha se recogen los siguiente atributos:



1. nombre de usuario.
2. hash de la password del usuario.
3. número de días desde que la password fue cambiada la última vez.

4. número de días antes de que la password pueda ser cambiada. 0 indica que en cualquier momento.
5. número de días máximo que la cuenta puede ser utilizada sin cambiar la password.
6. número de días a partir de los cuales notificar al usuario de que debe cambiar la password.
7. número de días a partir de los cuales la cuenta es bloqueada si la password ha expirado y no ha sido cambiada.
8. número de días desde el 1 de enero de 1970 que han transcurrido desde que la cuenta fue desactivada.
9. campo reservado para el futuro.

- **/etc/group**: recoger información sobre grupos. Utiliza el mismo patrón de formato que el resto de ficheros.

```
cdrom:x:24:vivek,student13,raj
```

1. nombre del grupo.
2. contraseña del grupo. X indica que el hash de la contraseña se encuentra en el fichero /etc/gshadow.
3. identificador numérico del grupo (GUID).
4. lista de nombres de usuarios separada por comas que son miembros del grupo.

- **/etc/gshadow**: recoge el hash de la password de los grupos, así como información sobre los miembros de los grupos e información de administración. Utiliza el mismo patrón de formato que el resto de ficheros.

```
general:!!:shelley:juan,bob
```

1. nombre del grupo.
2. hash de la password del grupo.
3. lista de nombres de usuarios separada por comas que son administradores del grupo (pueden añadir a otros usuarios y demás operaciones).
4. lista de nombres de usuarios separada por comas que pertenecen al grupo sin ser administradores.

- **/etc/sudoers.d**: contiene información sobre las políticas de uso del comando sudo. Para ajustar este fichero de forma segura es obligatorio el uso de la utilidad visudo, la cual verifica el correcto formato del fichero antes de guardar el archivo (para evitar disgustos). Mediante la modificación de este fichero, es posible por ejemplo permitir a un determinado usuario ejecutar un cierto número de comandos como root.

Documentación sobre el formato del fichero puede ser consultada aquí. Línea de ejemplo del fichero sudoers

```
joe ALL=(ALL) NOPASSWD: /full/path/to/command
```

Enlaces interesantes…

https://wiki.archlinux.org/index.php/users_and_groups
https://es.wikipedia.org/wiki/Shell_de_Unix
https://www.cyberciti.biz/faq/understanding-etcpasswd-file-format

2.2.2. Sistema de permisos de ficheros

Para poder trabajar con el sistema de permisos de Linux es necesario manejar los contextos sobre los que podemos aplicar permisos:

- **Usuario (u):** Se refiere al usuario propietario del fichero.
- **Grupo (g)**: Se refiere al grupo propietario del fichero.
- **Otros (o)**: Se refiere a los usuarios que no son el usuario propietario ni pertenecen al grupo propietario.
- **Todos (a)**: Se refiere a todos los contextos anteriores a la vez.

El sistema de permisos básico de Linux generalmente permite trabajar con tres niveles de permisos:

- **Escritura (w)**: Permite al contexto sobre el que se aplica modificar el fichero sobre el que se aplica el permiso.
- **Lectura (r)**: Permite al contexto sobre el que se aplica leer el contenido del fichero.
- **Ejecución (x)**: Permite al contexto sobre el que se aplica ejecutar el fichero.

Adicionalmente, se disponen de 3 permisos especiales:

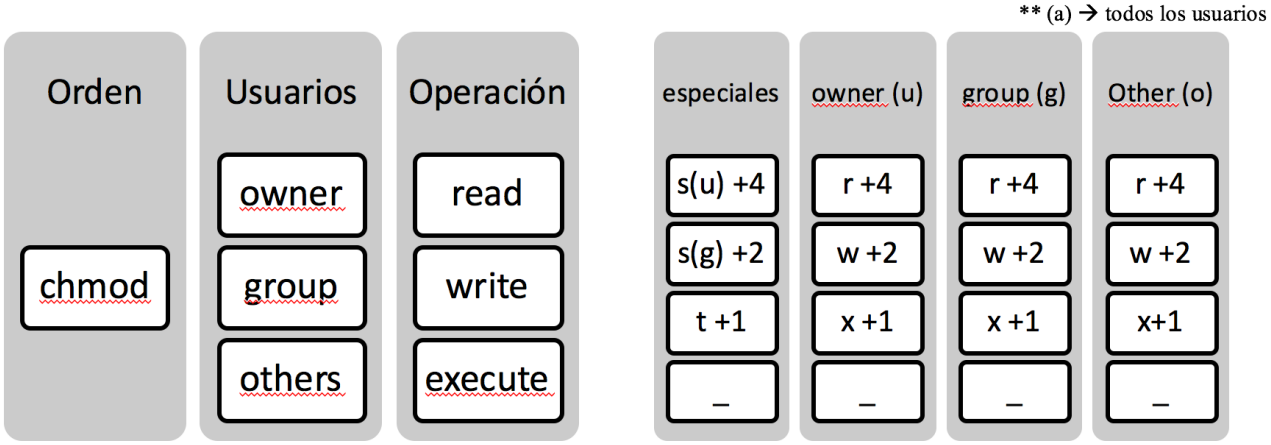
- **Setuid (SUID)**: Permite que un archivo ejecutable corra con los privilegios del propietario durante la ejecución a pesar de ser ejecutado por un usuario que podría ser diferente del propietario.
- **Setgid (SGID)**: Permite que un archivo ejecutable corra con los privilegios del grupo propietario durante la ejecución a pesar de ser ejecutado por un usuario que puede no pertenecer al grupo propietario.
- **Sticky bit**: El comportamiento del sticky bit difiere dependiendo del tipo de fichero. Si hablamos de un directorio, este permiso restringe a los usuarios a borrar, renombrar o mover el fichero (incluso si tienen permiso de escritura) a excepción del propietario o el

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

usuario root. Para el resto de ficheros ejecutables, cuando está activo el sticky bit, se le dice al sistema operativo que mantenga la imagen del proceso para ser reutilizado en posteriores ejecuciones. Esta funcionalidad tenía mucho sentido cuando los sistemas contaban de poca memoria, en la actualidad esta funcionalidad no aporta demasiado debido a la gran cantidad de memoria RAM con la que cuentan los computadores.

Es necesario tener en cuenta que el uso de los permisos SUID y SGID debe manejarse con cautela, puesto que, por ejemplo, en el caso de ficheros y ejecutables creados con el usuario **root**, estos pueden ser modificados para ejecutar acciones maliciosas con permisos de superusuario (en el módulo 8.4 se mostrará un ejemplo práctico al respecto)

Con la estructura anterior, los usuarios del sistema pueden definir los permisos de ficheros utilizando el comando chmod.



El comando chmod puede ser lanzado usando dos notaciones o modos distintos, octal (octal caracters) y texto claro (alphabetical characters).

- **<u>chmod modo Octal</u>**

Consiste en construir un número de 4 cifras en base 8 con la siguiente representación de izquierda a derecha.

- **[OPCIONAL] 1ª Cifra**: permisos especiales. Se parte de 0, si se quiere añadir el permiso de setuid, sumamos 4, si se desea añadir el permiso setgid, sumamos 2 y si se quiere añadir el permiso de sticky bit, sumanos 1. Ejemplo: un valor de 5 en la primera cifra significa que asignamos el permiso de setuid y el de sticky bit.
- **2ª Cifra**: permisos para el usuario propietario. Partimos de 0, si sumamos 4 añadimos el permiso de lectura, +2 permiso de escritura y si sumamos +1 el de ejecución.
- **3ª Cifra**: permisos para el grupo propietario. Partimos de 0, +4 para lectura, +2 escritura y +1 ejecución.

- **4ª Cifra**: permisos para el resto de los usuarios. De nuevo mismo formato, +4 lectura, +2 escritura y +1 ejecución.

Con esta representación se permite de forma fácil ajustar los permisos para los ficheros.
Algunos ejemplos:

- **755**: deja sin asignar los permisos especiales, para el usuario propietario asigna escritura, lectura y ejecución, para el grupo propietario, asigna lectura y ejecución y para el resto de usuarios, de nuevo asigna lectura y ejecución.
- **1644**: asigna el permiso especial setuid, para el usuario propietario asigna lectura y escritura, para el grupo propietario, asigna solo permiso de lectura y para el resto de usuarios de nuevo el permiso de lectura.

- **chmod modo texto**

Utiliza una representación basada en texto.
En primer lugar, se disponen de 3 operaciones. **+** añade un permiso, **-** quita un permiso, **=** asigna los permisos. Estas operaciones se pueden aplicar sobre los contextos vistos anteriormente representados por su letra (**a**: todos los contextos, **u**: usuario propietario, **g**: grupo propietario, **o**: resto de usuarios). Los permisos que se pueden añadir son los vistos anteriormente representados por su letra (**r**: lectura, **w**: escritura, **x**: ejecución, _: Indiferente). Con esto, se permiten crear las mismas combinaciones de permisos que con la representación octal. Cada operación realizada en un contexto va separada por comas.
Ejemplo:

- **u+x, g-w, o=rx**: esta combinación, añade el permiso de ejecución a los permisos que ya tenía el usuario, elimina el permiso de escritura a los permisos que ya tenía el grupo y asigna los permisos de lectura y ejecución sobre-escribiendo los que ya tenían el resto de usuarios.

Ejemplo de comando chmod para cambiar permisos a un fichero
chmod <PERMISOS> <PATH_DEL_FICHERO>

- chmod 775 /home/user
- chmod 0644 fichero.txt
- chmod o-rwx .ssh/id_rsa

Adicionalmente, podemos cambiar el usuario propietario y el grupo de un determinado fichero en nuestro sistema. Para realizar esto, utilizaremos el comando **chown** especificando quien es el usuario propietario, el grupo y el path del archivo sobre el que queremos aplicar el cambio.

Ejemplo de uso:
chown <USUARIO>:<GRUPO> <PATH_DEL_FICHERO>

- chown root:admin leeme.txt
- chown ubuntu:Ubuntu /home/ubuntu

Nota:
Podemos ver los permisos, así como el usuario propietario y grupo de un fichero con el comando **"ls -la"**

Enlaces interesantes…

https://www.linux.com/learn/understanding-linux-file-permissions
https://es.wikipedia.org/wiki/Chmod
https://es.wikipedia.org/wiki/Chown

3. FUENTES DEL SOFTWARE

Principalmente podemos clasificar el software que instalamos en nuestras máquinas en **software propio** (desarrollado internamente) y **software de terceros** (libre o propietario*)
Debemos tener ciertas cuestiones en cuenta en función del tipo de software que estamos instalando en nuestras máquinas.

Software propio

- Mayor responsabilidad. EJ: autoría de seguridad, QA, testeo, …
- Mayor control de qué y cómo hace su función el software.
- Acceso al código fuente.
- Gestión de tarballs, paquetes, versiones, etc.
- Repositorios internos con un mayor control sobre ellos.

Software de terceros

- Responsabilidad cae en manos de terceros (¡¡no nos exime de responsabilidad!!)
- Pérdida de control, controles de calidad, auditorias, qué hace en la máquina, …
- No siempre se tiene acceso al código.
- Gestión de tarballs, paquetes, versiones, etc.
- Repositorios externos, compartidos entre múltiples proyectos.

3.1. PAQUETES, GESTORES DE PAQUETES Y REPOSITORIOS

Independientemente de si el software es propio o de terceros, los siguientes conceptos son imprescindibles para el manejo de nuestro software.

Paquetes

Encapsula el software a instalar, ficheros de configuración, binarios, scripts, información de dependencias, versionado, etc. Existen paquetes autoejecutables u otros que requieren de software externo para ser instalados.

Gestor de paquetes

Sistema capaz de instalar paquetes en el sistema, se encarga de resolver las dependencias e instalarlas. En ocasiones, también ofrecen la posibilidad de descargar paquetes de los repositorios.

Repositorios

Realiza las labores de un almacén de paquetes de software. Los paquetes son identificados por un nombre cualificado, así como una o varias cadenas de versión. Es posible añadir repositorios como fuentes de paquetes a los gestores de paquetes, facilitándonos la búsqueda de software.

Enlaces interesantes...

https://en.wikipedia.org/wiki/Rpm_(software)
https://www.lifewire.com/guide-to-linux-packages-2202801
https://help.ubuntu.com/community/SoftwarePackagingFormats

3.2. ESTRUCTURA DE UN PAQUETE DEBIAN

Utilizaremos como referencia los paquetes .deb (Debian) aunque los conceptos base son extrapolables a otros tipos de paquetes Linux. El sistema operativo, Ubuntu usado como base para este módulo, es una distribución basada en Debian, por lo que usa estructura de paquetes por defecto en la gestión de paquetes de software.

```
paqueteEjemplo
|-- DEBIAN
|   |-- conffiles
|   |-- control
|   |-- md5sums
|   |-- postinst
|   `-- prerm
|-- debian-binary
|-- etc
|   |-- prueba2Conf.cfg
|   `-- pruebaConf.cfg
`-- usr
    `-- bin
        `-- paqueteEjemplo
            |-- binEjemplo
            `-- binEjemplo2
```

En la imagen de la izquierda, es posible ver la estructura de un paquete Debian típico. Dentro del paquete, partiendo de la raíz se deben replicar los path de destino de la máquina. El instalador, se encargará de posicionar los archivos en el lugar correcto durante la instalación del mismo. Ejemplo, en la imagen se puede apreciar un directorio etc que cuelga de la raíz y dentro contiene dos ficheros .cfg. Durante la instalación, el gestor de paquetes posicionará automáticamente esos ficheros en /etc. Adicionalmente, dentro del paquete se debe incluir un directorio llamado DEBIAN.

DEBIAN

Es imprescindible incluir un directorio llamado DEBIAN en el cual existen principalmente los siguientes ficheros:

- **conffiles**: fichero en texto plano que contiene la lista de cuáles son los ficheros de configuración.
- **control:** contiene los metadatos y la descripción del paquete.
- **md5sums:** contiene una lista con todos los path a los ficheros dentro del paquete y su md5.
- **pretinst:** script shell con las instrucciones a realizar al principio de la instalación.
- **postinst:** script shell con las instrucciones a realizar cuando finaliza la instalación.
- **prerm:** script shell con las instrucciones a realizar antes de desinstalar el paquete.
- **postrm:** script shell con las instrucciones a realizar después de desinstalar el paquete.

Enlaces interesantes...

https://en.wikipedia.org/wiki/Deb_(file_format)
https://ubuntulife.wordpress.com/2010/08/05/crear-un-paquete-deb-manualmente/

3.3. ¿QUÉ HACE ESTE PAQUETE DEBIAN?

Es **fundamental** controlar el software que instalamos en nuestras máquinas. Para tener el mayor control, sobre todo en **software de terceros**, es importante responder a la pregunta: ¿Qué hace este paquete en mi máquina al ser **instalado**?

Ahora que conocemos la estructura de un paquete Debian, podemos observar los siguientes scripts para hacernos una idea de qué es lo que hace este paquete en nuestra máquina: **preinst, postinst, prerm, postrm**.

Además, también es muy interesante **evaluar los ficheros que se van a copiar a la máquina** desde el paquete. Esto es posible verlo analizando la estructura del paquete, aunque **si el paquete ya ha sido instalado** es posible obtenerlo utilizando el **gestor de paquetes**.

- **¿cómo podemos descargar un paquete Debian?** En el ejemplo el paquete es htop, la versión 1.0.2-3.

```
apt-get download htop=1.0.2-3
```

- **¿cómo podemos ver el contenido del paquete descargado?**

```
dpkg-deb -R htop_1.0.2-3_amd64.deb salida
```

- **¿cómo podemos ver los ficheros que un paquete ha modificado en nuestro sistema al haberlo instalado?** También podemos ver la estructura interna del paquete.

```
dpkg -L htop
```

3.4. GESTORES DE PAQUETES (APT)

En los sistemas Linux existen numerosos gestores de paquetes, normalmente asociados a la distribución con la que se trabaja.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

La lista de los principales gestores de paquetes para los sistemas Linux puede consultarse aquí:

https://en.wikipedia.org/wiki/List_of_software_package_management_systems

En nuestro caso vamos a centrarnos en **APT (Advanced Packaging Tool).**

comando	Descripción
apt list --installed	Muestra la lista de paquetes instalados tanto manual como automáticamente, así como sus versiones y si son actualizables a una versión superior
apt-cache show PAQUETE	Muestra la información del paquete
apt-cache showpkg PAQUETE	Muestra las dependencias del paquete
apt-cache stats PAQUETE	Estadísticas del paquete
apt-get update	Actualiza la lista de paquetes disponibles de las fuentes especificadas de repositorios
apt-get upgrade	Aplica actualizaciones a todo el software instalado en el sistema
apt-get install PAQUETE	Instala el paquete especificado
apt-get remove PAQUETE	Desinstala el paquete conservando los ficheros de configuración
apt-get purge PAQUETE	Desinstala el paquete incluyendo los ficheros de configuración
apt-get check	Comprueba si hay dependencias rotas

Más información aquí.

3.5. GESTIÓN DE REPOSITORIOS SOFTWARE (SOURCES)

Una labor importante es verificar las fuentes de software (especialmente si las fuentes son de

terceros). En nuestro caso continuaremos con APT, aunque los conceptos se pueden extrapolar. Algunas cuestiones a tener en cuenta…

- **¿Dónde se almacenan los repositorios?** En /etc/apt/sources.list
- **¿Cómo se añaden nuevas fuentes?** Añadiendo una nueva línea en el fichero anterior. También se puede usar la herramienta add-apt-repository.
- **¿Cómo se eliminan fuentes?** Eliminando la línea correspondiente.
- **¿Cómo se sincronizan los índices con el fichero tras un cambio?** apt-get update

Enlaces interesantes…

https://help.ubuntu.com/community/Repositories/CommandLine
http://manpages.ubuntu.com/manpages/trusty/man1/add-apt-repository.1.html

3.6. GESTIÓN DE VULNERABILIDADES CONOCIDAS

Es importante gestionar en todo el **software instalado (incluyendo dependencias)** en nuestras máquinas qué **vulnerabilidades conocidas** existen. Resulta intratable realizar esta labor manualmente en una sola máquina e **imposible cuando se dispone de una granja de ellas**. Por lo anterior, es importante utilizar software adecuado para realizar este tipo de tareas.

Como se ha comentado en otros módulos de este máster, existen **bases de datos de vulnerabilidades conocidas** que pueden ser consultadas para verificar nuestro software. Algunas de estas bases de datos están incluidas en los siguientes enlaces enlaces:

https://nvd.nist.gov/
https://vuldb.com/
https://www.owasp.org/index.php/Main_Page
https://www.spiceworks.com/free-software-inventory-audit-tool

Una forma sencilla de obtener la lista de paquetes instalados por el usuario es mediante el comando apt list --installed, obteniendo además las versiones de cada uno y e información sobre las últimas versiones disponibles de los mismos.

```
xserver-xorg-video-amdgpu/focal,now 19.1.0-1 amd64 [instalado, automático]
xserver-xorg-video-ati/focal,now 1:19.1.0-1 amd64 [instalado, automático]
xserver-xorg-video-fbdev/focal,now 1:0.5.0-1ubuntu1 amd64 [instalado, automático]
xserver-xorg-video-intel/focal,now 2:2.99.917+git20200226-1 amd64 [instalado, automático]
xserver-xorg-video-nouveau/focal,now 1:1.0.16-1 amd64 [instalado, automático]
xserver-xorg-video-qxl/focal,now 0.1.5+git20200331-1 amd64 [instalado, automático]
xserver-xorg-video-radeon/focal,now 1:19.1.0-1 amd64 [instalado, automático]
xserver-xorg-video-vesa/focal,now 1:2.4.0-2 amd64 [instalado, automático]
xserver-xorg-video-vmware/focal,now 1:13.3.0-3 amd64 [instalado, automático]
xserver-xorg/focal,now 1:7.7+19ubuntu14 amd64 [instalado, automático]
xul-ext-ubufox/focal,focal,now 3.4-0ubuntu1.17.10.1 all [instalado, automático]
xwayland/focal-security,now 2:1.20.8-2ubuntu2.6 amd64 [instalado, actualizable a: 2:1.20.9-2ub
untu1.1~20.04.1]
xxd/focal,now 2:8.1.2269-1ubuntu5 amd64 [instalado, automático]
xz-utils/focal,now 5.2.4-1 amd64 [instalado, actualizable a: 5.2.4-1ubuntu1]
yaru-theme-gnome-shell/now 20.04.7 all [instalado, actualizable a: 20.04.10.1]
yaru-theme-gtk/now 20.04.7 all [instalado, actualizable a: 20.04.10.1]
yaru-theme-icon/now 20.04.7 all [instalado, actualizable a: 20.04.10.1]
yaru-theme-sound/now 20.04.7 all [instalado, actualizable a: 20.04.10.1]
yelp-xsl/focal,focal,now 3.36.0-1 all [instalado, automático]
yelp/focal,now 3.36.0-1 amd64 [instalado, automático]
zenity-common/focal,focal,now 3.32.0-5 all [instalado, automático]
zenity/focal,now 3.32.0-5 amd64 [instalado, automático]
zerofree/focal,now 1.1.1-1 amd64 [instalado, automático]
zip/focal,now 3.0-11build1 amd64 [instalado, automático]
zlib1g/focal,now 1:1.2.11.dfsg-2ubuntu1 amd64 [instalado, actualizable a: 1:1.2.11.dfsg-2ubunt
u1.2]
```

Ejemplo de salida del comando apt list --installed

Una alternativa es el uso de la herramienta **apt-mark**, que mediante los comandos apt-mark showmanual y apt-mark showauto nos devolverá la lista de paquetes instalados en el sistema, permitiendo diferenciar cuáles de estos han sido instalados de forma manual o automática.

Listar aquellas versiones disponibles que correspondan con actualizaciones de seguridad es posible ejecutando el comando

```
apt list --upgradable | grep "\-security"
```

Una forma sencilla de realizar las actualizaciones de seguridad disponibles es mediante el uso de la herramienta **unattended-upgrade**, pudiendo realizar una simulación de instalación con el parámetro **--dry-run**, que simplemente realizará una simulación de ejecución sin descargar ni instalar nada con el objetivo de comprobar que no existan problemas (por ejemplo de obtención de dependencias) con la instalación real.

3.7. GESTIÓN DE PAQUETES EN OTRAS DISTRIBUCIONES LINUX.

3.7.1. Gestor de paquetes ARP.

RPM Package Manager (o **RPM**, originalmente llamado ***Red Hat Package Manager***, pero se convirtió en acrónimo recursivo) es una herramienta de administración de paquetes pensada básicamente para GNU/Linux. Es capaz de instalar, actualizar, desinstalar, verificar y solicitar programas de la misma forma en la que lo hace APT en el caso de distribuciones basadas en Debian. RPM es el formato de paquete de partida del Linux Standard Base.

Originalmente desarrollado por Red Hat para Red Hat Linux, en la actualidad muchas distribuciones GNU/Linux lo usan, dentro de las cuales las más destacadas son Fedora, CentOS, Mandriva, Mageia, PCLinuxOS, openSUSE, SuSE Linux. También se ha portado a otros sistemas operativos. De hecho, es posible instalarlos en distribuciones basadas de Debian, como por ejemplo Ubuntu, aunque no de forma nativa. Mas información: https://ubunlog.com/como-instalar-paquetes-rpm-en-ubuntu-y-sus-derivados/

En el caso de RedHat, la gestión de paquetes usando RPM se hace mediante el comando rpm. Así, podremos instalar nuevos paquetes usando la opción rpm -i, así como rpm -U para actualizar uno en concreto. Algunos comandos útiles:

rpm -qa: Mostrar paquetes instalados.
rpm -qi foo: Mostrar la información de un paquete RPM.
rpm -ql foo: Listar ficheros de un paquete RPM instalado.
rpm -qc foo: Listar solo los ficheros de configuración.
rpm --checksig foo: Verificar firma de un paquete RPM.
rpm -ivh "localFile.rpm": Instala run paquete.
rpm -e "localFile.rpm": Desinstalar un paquete.

Un gestor adicional de paquetes RPM es yum. Sus comandos son muy similares a los de APT. A continuación se muestran los comandos principales de yum:

yum install package: Instalar un paquete.
yum remove package: Desinstalar un paquete.
yum update package: Actualizar un paquete.

Mas info y comandos: https://access.redhat.com/es/solutions/238003

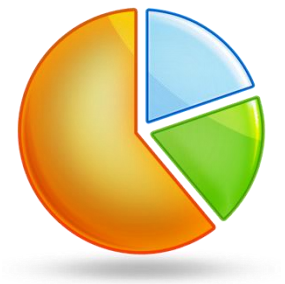
3.7.2. Diferencias entre paquetes ARP y DEBIAN

Una de las principales diferencias entre los paquetes .rpm y .deb, es que a la hora de actualizar un paquete, los .rpm no necesitan incluir los mismos datos que el instalador original. Es decir, si se desea, solo se tendrán que incluir los archivos a actualizar, lo que reduce significativamente el peso de los paquetes a descargar.

3.7.3. Otros formatos de paquetes usados en Linux

Además de .deb y .rpm existen multitud de formatos de paquetes software minoritarios en Linux además de gestores asociados a estos. En el siguiente enlace se enumeran algunos interesantes: https://es.wikipedia.org/wiki/Formatos_de_paquetes_en_Linux

4. GESTIÓN Y LIMITACIÓN DEL USO DE RECURSOS



En una máquina, ejecutándose en un mismo sistema operativo, **conviven** un gran número de procesos de diferentes usuarios, que comparten los recursos disponibles unos con otros. Aun asumiendo en la buena fe de cada uno de ellos (usuarios y procesos), los procesos van a competir por el acceso a estos recursos, pudiendo llegar a producirse situaciones en las cuales algunos procesos pueden monopolizar el uso de recursos haciendo que el resto no puedan funcionar con normalidad.

Al margen de la situación anterior, donde se asumía la buena fe de los procesos y usuarios que están utilizando la máquina, pensemos ahora en un escenario en el que existe un proceso que está siendo atacado y que por supuesto, convive con otros procesos en la máquina… Imaginemos por ejemplo que ese proceso expone un servicio web y que un atacante ha identificado una determinada operación que le permite provocar un aumento en el uso de memoria en la máquina. En este escenario, el atacante decide automatizar un sistema que repite esa petición una y otra vez con la intención de cómo mínimo consumir toda la memoria posible hasta dejar la máquina inutilizada. También existe la posibilidad de que se quiera llevar la máquina a un estado de carga tal que determinados mecanismos de seguridad puedan dejar de funcionar y aprovechar esta situación para entrar en la misma o realizar otro tipo de ataques. Este patrón de ataque puede ser aplicado sobre cualquier recurso, véase uso de CPU, uso de

disco, uso de red, etc.

Independientemente de la fuente del problema, seguridad o competitividad por los recursos de los procesos, los administradores de sistemas operativos Linux cuentan con herramientas que el propio kernel ofrece para lidiar con este tipo de problemas.

Un ejemplo es el comando: ulimit que permite modificar los limites de recursos que un proceso puede usar. Así, utilizando **ulimit -s** permite definir el tamaño de pila que podrá usar el proceso, permitiendo ampliarlo por encima del tamaño por defecto de 8M si por ejemplo estamos trabajando con procesos con una alta recursión.

Otro ejemplo de uso de este comando es mediante la opción **ulimit -v**, que permite definir el tamaño máximo de memoria virtual que usarán los procesos.

Estas restricciones también podrían ser aplicadas a grupos de procesos utilizando el concepto de Linux control groups (cgroups) que será explicado más adelante.

Algunas preguntas relacionadas a las cuales un administrador debe hacer frente son las siguientes:

- ¿podría un atacante cambiar el comportamiento de un proceso para **monopolizar el uso de CPU**?
- ¿y hacer que el **consumo de memoria** de ese proceso aumente tanto que otros servicios mueran por falta de memoria?
- ¿y hacer que el número de **conexiones abiertas** aumente tanto que no se puedan atender más?
- ¿qué hay del número de **ficheros abiertos**?
- ¿podría aumentar el consumo de **espacio en disco**?

Enlaces interesantes...

http://manpages.ubuntu.com/manpages/xenial/man5/limits.conf.5.htm
http://manpages.ubuntu.com/manpages/trusty/man1/ulimit.1posix.html
http://manpages.ubuntu.com/manpages/xenial/man8/pam_limits.8.html

4.1. EL DIRECTORIO /ETC/SECURITY

En las distribuciones Linux, podemos encontrar un directorio bajo el path /etc/security, en el que podemos encontrar diferentes archivos encargados de gestionar algunas de las políticas de seguridad de nuestro sistema operativo. Veamos qué elementos podemos encontrar en su interior.

- **access.conf**: Fichero de configuración de la tabla de control de acceso. En él, es posible definir para que pareja de usuario y host, en el caso de accesos remotos o pareja de usuario y tty se permiten o no hacer login en nuestro sistema.
- **capability.conf**: Linux, con el propósito de evitar que los procesos deban ser ejecutados con permisos root, permite asignar capacidades a usuarios convencionales que inicialmente pertenecen al usuario root. Para hacer esto de forma precisa, los sistemas operativos Linux ofrecen una lista de funcionalidades o capacidades que pueden ser heredadas por los usuarios convencionales. La lista de estas capacidades puede ser encontrada aquí. En el fichero capability.conf se permite definir que capacidades se conceden a qué usuarios. El formato del fichero se puede consultar aquí.
- **group.conf**: Hasta ahora, cuando vimos el sistema de usuarios de Linux, abordamos que estos podían pertenecer a grupos y en base a los grupos de usuarios a los que pertenezcan van a tener permiso para acceder a determinados ficheros o incluso a determinadas acciones dentro del sistema, además de los permisos del propio usuario en sí. Hasta aquí todo bien, pero que ocurre cuando la pertenencia a un determinado grupo (y por consiguiente el acceso a los permisos) debe estar supeditada a determinadas condiciones. Por ejemplo, imaginemos que disponemos de un sistema al que los empleados acceden durante la jornada laboral, tal vez tendría sentido que los usuarios dispongan de los permisos que les concede el grupo de empleados solo durante el período de tiempo que dura su jornada. Por otro lado, tendría sentido que la pertenencia a un determinado grupo dependiera del servicio que vas a usar, por ejemplo, si vas a usar servicios orientado a realizar tu actividad laboral, podría tener sentido que pertenezcas al grupo empleados mientras que durante la utilización de otros servicios no relacionados, no tiene sentido que se siga disponiendo de los mismos permisos asociados al grupo empleados. Es decir, se puede ajustar la pertenencia a un determinado grupo y por tanto heredar sus permisos en base al servicio que se esté usando. Este es el propósito del fichero de configuración group.conf, el poder definir reglas que permitan ajustar la pertenencia a un grupo dinámicamente en función de determinadas condiciones temporales o de uso.
- **namespaces***: Recoge toda la gestión de mecanismos relacionados con la poli-instanciación de recursos del sistema operativo. Esto será abordado específicamente más adelante en el temario. Evita que el uso de un recuso por un determinado proceso impacte en el uso que está haciendo otro proceso, esto se consigue mediante la generación de una instancia nueva del recurso para cada proceso.
- **opasswd**: Fichero dedicado al almacenamiento de passwords antiguas que han sido utilizadas por los usuarios. El propósito de este fichero es evitar que los usuarios repitan contraseñas antiguas cuando cambian de password.
- **pam_env.conf**: Fichero donde se pueden declarar variables de entorno que estarán disponibles para todos los usuarios del sistema. Información al respecto puede ser consultada aquí.

- **time.conf**: En este fichero se pueden definir reglas para restringir el login a usuarios basándose en rangos temporales. Ejemplo, desactivar el acceso al sistema a empleados durante los fines de semana. Documentación adicional puede ser encontrada aquí.
- **limits.conf**: Permite establecer límites sobre el uso de recursos en el sistema. Cómo utilizar este fichero se aborda en el siguiente apartado.

4.2. ESTABLECIENDO LÍMITES DE USO DE RECURSOS

Podemos ver la estructura de cada línea del fichero de configuración en la siguiente tabla. Por cada línea del fichero **/etc/security/limits.conf** se define una regla que establece un límite. Cada límite se aplica sobre un dominio, entendiéndose por dominio a un usuario, grupo, o todos los usuarios o todos los grupos. Existen dos tipos de límites que se pueden establecer según el nivel de restricción o bloqueo que apliquen al proceso: Soft o Hard. Cuando un límite es Soft, el límite puede ser excedido durante un período de tiempo especificado mientras que si el límite es Hard, éste no podrá ser excedido nunca. El límite se debe aplicar sobre un recurso, el cual debe ser identificado por un identificador propio.

En la siguiente tabla se puede ver el formato de cada línea del fichero limits.conf.

	Dominio	Tipo	Item	Valor
Descripción	- **userName** - **@groupName** - ***/%** → Cualquier usuario / grupo - ***El usuario root no es limitado por */%	- **soft**: Este tipo de límite puede ser excedido temporalmente. - **hard**: Este tipo de límite no puede ser excedido nunca.	Recurso sobre el que se aplica el límite	Límite físico a establecer en el recurso
Ejemplos	admin usuario1 @consultants @developers	soft hard hard soft	nproc maxlogins nofile as	2 100 1000 256

Comando interesante... ulimit –a

4.3. LISTA DE RECURSOS A LIMITAR EN /ETC/SECURITY/LIMITS.CONF

Item	Descripción
core	Máximo tamaño de ficheros core
data	Máximo tamaño datos
fsize	Máximo tamaño de fichero
memlock	Máximo tamaño memoria reservada
nofile	Número de ficheros abiertos
rss	Máximo tamaño en memoria residente
stack	Máximo tamaño de pila
cpu	Máximo tiempo de CPU
nproc	Máximo número de procesos
as	Espacio de memoria
maxlogins	Máximo número de logins
maxsyslogins	Máximo número de logins en el sistema
priority	Prioridad de los procesos que el usuario ejecute
locks	Máximo número de ficheros bloqueados
sigpending	Máximo número de señales pendientes
msgqueue	Max. Memoria usada en colas de mensajes
nice	Max. nice priority
rtprio	Max. realtime priority

Cabe destacar, que para poder aplicar límites en el sistema, es necesario activar la gestión de límites modificando el fichero /etc/pam.d/common-session y añadiendo al final del mismo la línea "session required pam_limits.so". Tras esto, debemos reiniciar el sistema y los límites definidos en el fichero /etc/security/limits.conf serán tenidos en cuenta.

4.4. ACOTANDO EL USO QUE UN PROCESO HACE DE LA CPU

Existen dos elementos que se pueden acotar con respecto a la CPU:

- **Tiempo que el proceso va a utilizar la CPU**: Para poder acotar el tiempo que un proceso va a recibir de CPU, hay que tener en cuenta un concepto como el planificador. Como se comentó en la introducción, el planificador es el encargado de asignar el recurso de la CPU a cada proceso. Para cada proceso, es posible asignar un valor de prioridad que el planificador tendrá en cuenta de cara a asignar la CPU al proceso. Cuanto más bajo es el valor de prioridad asignado, mayor probabilidad tiene el proceso de recibir el recurso de CPU, es decir, en igualdad de condiciones, cuando dos procesos están compitiendo por el uso de CPU, si uno tiene prioridad 10 y otro tiene prioridad 20, el primero en obtener acceso a la CPU será el proceso de prioridad 10.
- **Porcentaje de utilización de la CPU**: Una vez un proceso tiene asignada la CPU, también es posible acotar el porcentaje de utilización que el proceso hace del recurso. Esta limitación no suele tener fines de encapsulación o de control de recursos sino para propósitos de testeo y depuración.

Comando	Descripción
cpulimit –l 20 –p 1234 -b	Ajusta el porcentaje de uso máximo de CPU al 20% para el proceso cuyo PID es 1234
nice –n 10 stress –c 1	Ejecuta el comando "stress –c 1" con prioridad nice 10. Mayores recursos se asignan a procesos con valor más bajo.

PID: identificador de proceso.

PLANIFICADOR:
Encargado de **gestionar el acceso** de los procesos **a la CPU**. Debe hacer un reparto justo entre procesos evitando que algunos monopolicen el acceso.

NOTA:
Explicar el funcionamiento del planificador del sistema operativo excede el alcance de este Módulo, aunque se puede encontrar información adicional aquí.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Enlaces interesantes…
http://manpages.ubuntu.com/manpages/xenial/man1/cpulimit.1.html
http://manpages.ubuntu.com/manpages/trusty/es/man1/nice.1.html
https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/6/html/performance_tuning_guide/s-cpu-scheduler
http://manpages.ubuntu.com/manpages/trusty/man1/stress.1.html

4.5. CUOTAS DE DISCO

A lo largo de esta sección, hemos ido abordando límites que afectan a la mayoría de recursos a excepción de las cuotas de uso de disco.
De forma nativa, no es posible establecer límites tales como cuanto espacio un usuario o grupo de usuarios pueden consumir en una determinada unidad, por lo que para poder utilizar esta funcionalidad, al igual que se hizo con las ACL, debemos activarla en el punto de montaje del sistema de ficheros sobre el que se desea aplicar las cuotas de disco. Para activar el control de cuota es necesario en primer lugar instalar el paquete **quota** con nuestro gestor de paquetes habitual. Tras esto, es necesario modificar el fichero **/etc/fstab** como se indicó en la sección 3.1 de este módulo para añadir las opciones **usrquota** y **grpquota**, las cuales activan la gestión de cuota a nivel de usuario y grupo para el sistema de ficheros indicado. Finalmente, tras realizar la modificación en el fichero, es necesario remontar el sistema de ficheros utilizando el comando **mount –a** o reiniciando la máquina.

En el fichero **/etc/mtab**, podemos encontrar un listado de los puntos de montaje junto con la configuración aplicada. Este fichero puede ser utilizado para comprobar que las opciones de montaje han sido aplicadas correctamente.

Una vez activada la limitación por cuota en un sistema de ficheros, en la raíz del punto de montaje aparecerán dos ficheros llamados **aquota.user** y **aquota.group** sobre los cuales se apoyará el sistema de control de cuotas para la gestión de las mismas.

Podemos utilizar los siguientes comandos para administrar nuestras cuotas de disco.

- **setquota**: Establece una cuota de disco.
- **edquota**: Edita las cuotas de disco de un usuario o grupo.
- **quota**: Muestra las cuotas de disco y los límites.
- **quotacheck**: Escanea un sistema de archivos para comprobar el uso de disco y crea y repara los ficheros de gestión de cuota.
- **quotaon**: Activa la gestión de cuotas en un sistema de ficheros.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **quotaoff**: Desactiva la gestión de cuotas en un sistema de ficheros.
- **repquota**: Muestra la información resumida de las cuotas de un determinado sistema de ficheros.
- **warnquota**: Envía emails a los usuarios sobre las cuotas.

Principalmente, existen dos tipos de cuotas que podemos aplicar a un usuario o grupo de usuarios en un sistema de archivos:

- **Basado en número de ficheros**: Este tipo de cuota monitoriza el número de ficheros que un usuario o grupo de usuarios han creado en el disco para establecer límites sobre ello.
- **Basado en uso de espacio en disco**: Monitoriza el espacio consumido por los archivos pertenecientes a un usuario o grupos de usuarios para establecer límites en base a ello.

De forma similar al resto de límites vistos anteriormente, existen dos tipos de cuotas en base a como de estricto es el sistema en cuanto al cumplimiento de la misma:

- **Soft**: El valor de la cuota establecido puede ser sobrepasado durante un tiempo de gracia el cual es configurable por el administrador de cuotas. Una vez el tiempo de gracia ha expirado, el usuario ya no puede crear más ficheros o utilizar más espacio en disco hasta que no reduzca el uso por debajo del límite.
- **Hard**: El valor establecido de la cuota nunca podrá ser sobrepasado por el usuario. En caso de que este intente hacerlo recibirá un mensaje de error.

Enlaces interesantes…

http://manpages.ubuntu.com/manpages/zesty/man8/setquota.8.html
http://manpages.ubuntu.com/manpages/zesty/man8/edquota.8.html
http://manpages.ubuntu.com/manpages/zesty/man1/quota.1.html
http://manpages.ubuntu.com/manpages/zesty/man8/quotacheck.8.html
http://manpages.ubuntu.com/manpages/zesty/man8/quotaon.8.html
http://manpages.ubuntu.com/manpages/zesty/man8/repquota.8.html

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

5. GESTIÓN DE REDES EN SISTEMAS LINUX

En este bloque se tratarán de abordar algunos de los componentes orientados a la gestión de redes que suelen ser comúnmente encontrados dentro de una instancia Linux, muchos de ellos orientados a la gestión de la propia red y otros, en cambio, orientados a tareas tales como control del tráfico y protección.

El objetivo de esta sección no es profundizar en todos ellos, ya que daría para otro módulo. En lugar de ello, da un repaso a vista de pájaro de los componentes más comunes.

5.1. SERVICIOS MÍNIMOS

Existen unos servicios mínimos que necesita un sistema para conectarse a Internet. Uno de ellos, **NTP** (Network Time Protocol), se utiliza para que los sistemas se sincronicen entre ellos. Por otro lado, **DNS** (Domain Name System) es un servicio imprescindible de Internet, que mantiene registro de los nombres de dominios y su dirección IP.

5.1.1. NTP

Es un protocolo de Internet para sincronizar los relojes de los sistemas informáticos a través del enrutamiento de paquetes en redes con latencia variable.

El demonio NTP de Unix es un proceso de nivel de usuario que se ejecuta continuamente en la máquina que soporta NTP y la mayor parte del protocolo está implementado en este proceso de usuario.

Para obtener el mejor rendimiento de NTP, es importante tener un reloj NTP estándar con lazo de seguimiento de fase implementado en el kernel del Sistema operativo, en vez de sólo usar la intervención de un demonio NTP externo: todas las versiones actuales de GNU/Linux y Solaris soportan esta característica.

Dependiendo de la distribución de Linux, el programa NTP se configura utilizando el archivo **/etc/ntp.conf** o **/etc/xntp.conf**.

El archivo ntp.conf más básico tendrá listados dos servidores, uno es con el que se desea sincronizar y una pseudo IP. Éste último se utiliza en caso de problemas de red o si el servidor NTP remoto sufre una caída.

Se recomienda listar al menos dos servidores remotos con los que poder sincronizarse. Uno será el primario y el otro se utilizará como backup.

Además, el archivo ntp.conf debe tener una línea que indica dónde está el driftfile, un archivo donde se encuentra guardado el error de frecuencia del reloj del sistema. El archivo se actualiza una vez cada hora por ntpd (daemon NTP).
A continuación, se expone un ejemplo de un archivo ntp.conf:

```
# --- GENERAL CONFIGURATION ---
server  127.127.1.0
fudge   127.127.1.0 stratum 10
# Drift file.
driftfile /etc/ntp/drift

# Se puede prohibir el acceso general a este servicio:
restrict default ignore

# Es posible permitir que sistemas de una red sincronicen con el servicio
# pero no lo modifiquen:
restrict 72.12.12.42 nomodify

# También se puede permitir acceso no restringido a ntpd:
restrict 127.0.0.1
```

Ataque Delorean:
Existen ataques que utilizan NTP para lograr su fin. Por ejemplo, el ataque Delorean realiza un ataque Man in the Middle para enviar al cliente una fecha diferente a la que envió el servidor original y así permitir explotar vulnerabilidades que sólo serían aplicables en un determinado rango de tiempo, por ejemplo, la validez de un certificado o alguna directiva de seguridad como por ejemplo la cabecera HSTS.

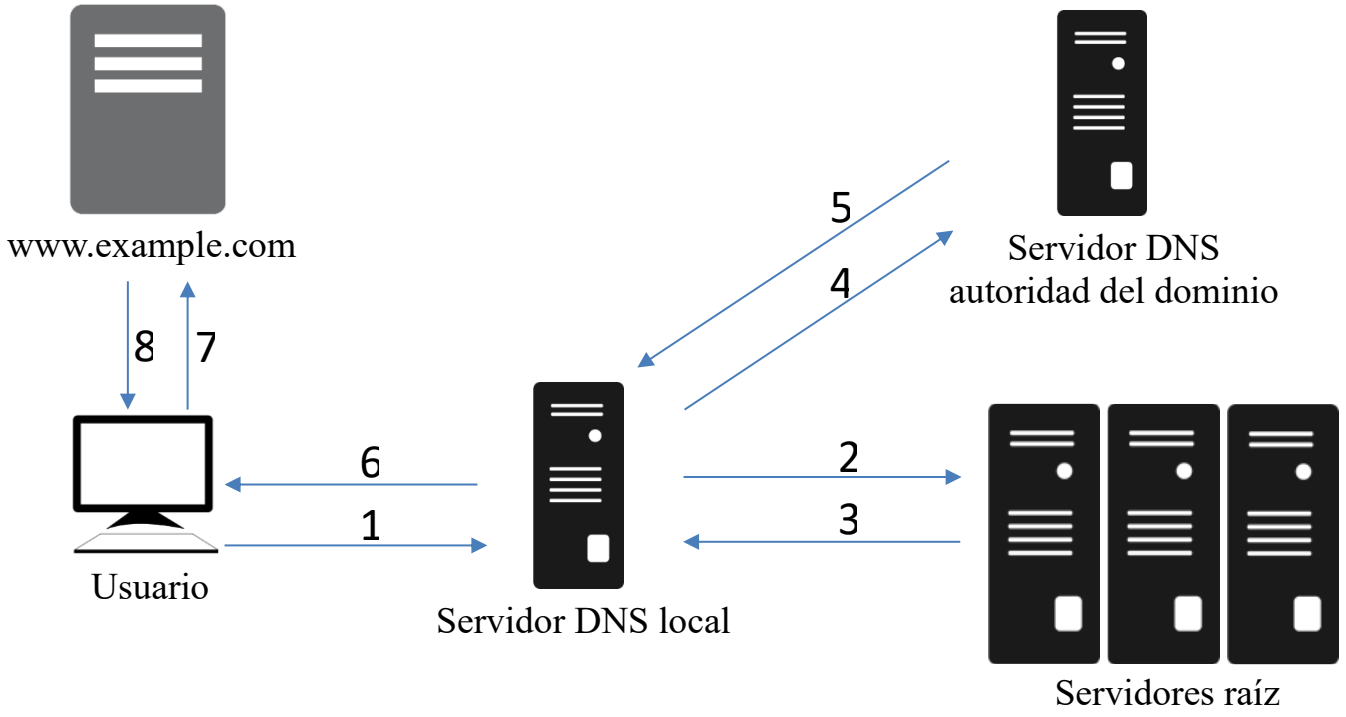
5.1.2. DNS

Cuando se visita una web, normalmente se utiliza un nombre de dominio, mucho más fácil de recordar para un humano, en cambio no muy útil para un computador. Para solucionar este problema, existen los servidores DNS, encargados de convertir el dominio en IPs. Cualquier petición generada en un computador, es redirigida a un servidor de nombres de dominio (DNS) el cual responde con una dirección IP. Tras esto, la petición sigue su curso hacia la IP del servidor de destino.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Cuando un sistema quiere acceder a un host, ocurren las siguientes fases:

a) El sistema local revisa su caché interna de DNS para resolver la dirección IP. Si no se encuentra, el usuario envía de nuevo la consulta al servidor DNS local (configurado en el host).
b) El servidor DNS local revisa su caché en búsqueda de la dirección IP. Si no la encuentra, consulta a los servidores raíz cuál es el servidor autoridad del dominio en cuestión.
c) El servidor raíz responde con la información necesaria del servidor autoridad.
d) El servidor DNS local ahora pregunta directamente por la dirección IP, por ejemplo www.example.com, al servidor DNS autoridad del dominio example.com, el cual debe tener todos los registros DNS del dominio.
e) El servidor autoridad del dominio example.com responde a la consulta entregando la dirección correspondiente para el host.
f) El servidor DNS local almacena esta respuesta para futuras consultas, y entrega la dirección IP al computador que originó la consulta.
g) El computador también almacena la dirección IP en su caché. Luego, el navegador web genera una solicitud HTTP para el host del dominio example.com, el cual se encuentra en la IP devuelta por el servidor DNS.
h) El servidor remoto, responde a la solicitud y entrega la página al navegador web.



Existen varios archivos interesantes que afectan a la configuración de DNS:

- **/etc/resolv.conf**: Contiene al menos una línea con un nameserver. Cada línea de nameserver define un servidor DNS. Por ejemplo:

```
# Google IPv4 nameservers
nameserver 8.8.8.8
nameserver 8.8.4.4
```

- **/etc/hosts**: Se utiliza para traducir hostnames en direcciones IP. El sistema buscará en este archivo cuando se quiera acceder a un hostname. Para añadir una entrada, se escribe el hostname y la IP correspondiente en la misma línea, separados por una tabulación.

```
127.0.0.1	localhost
```

- **/etc/sysconfig/network**: Se utiliza para especificar información sobre la configuración de la red. Contiene las variables NETWORKING, HOSTNAME, GATEWAY, GATEWAYDEV, NISDOMAIN y NOZEROCONF. Por ejemplo:

```
TYPE=Ethernet
BOOTPROTO=none
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV4_FAILURE_FATAL=no
NAME=eth0
UUID=...
ONBOOT=yes
HWADDR=0e:a5:1a:b6:fc:86
IPADDR0=172.31.24.10
PREFIX0=23
GATEWAY0=172.31.24.1
DNS1=192.168.154.3
DNS2=10.216.106.3
DOMAIN=example.com
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
```

5.2. FIREWALL

Después de estudiar el DNS y NTP, se presenta el firewall, otro elemento vital en la seguridad de un sistema.
El firewall es un sistema de seguridad de red que monitoriza y controla el tráfico entrante y saliente basándose en determinadas reglas de seguridad.

Linux tiene en su kernel el firewall iptables, incluido desde la versión 2.4.

Iptables es un programa que permite a un administrador del sistema configurar las tablas del firewall del kernel de Linux y las cadenas y reglas que guarda. Requiere privilegios elevados para operar y debe ejecutarse por el usuario root, en otro caso, la función fallará.

5.2.1. Comandos básicos

A continuación, se exponen algunos comandos útiles de iptables:

Iniciar el servicio firewall:
```
sudo service iptables start
```

Detener el servicio firewall:
```
sudo service iptables stop
```

Reiniciar el servicio firewall:
```
sudo service iptables restart
```

Agregar una regla a la cadena especificada:
```
sudo iptables -A <cadena> <regla>
```

Eliminar una regla de una cadena especificada:
```
sudo iptables -D <cadena> <regla>
```

Borra todas las reglas de una cadena (todas las cadenas si no se especifica):
```
sudo iptables -F <cadena>
```

El protocolo de la regla o del paquete para verificar (tcp, udp, udplite, icmp, esp, ah, sctp, all):
```
sudo iptables -p <protocolo>
```

5.2.2. Filtrado de tráfico

Una de las funciones del Firewall es el filtrado de tráfico. Para permitir el tráfico procedente de un determinado rango de IPs, hay que ejecutar el siguiente comando:

```
sudo iptables -A INPUT -p tcp -m iprange -src-range 192.168.1.13-192.168.2.19
```

También sería posible permitir por dirección MAC:

```
sudo iptables -A INPUT -m mac -mac-source 00:00:00:00:00:01
```

Para crear una regla que permite el tráfico a través de un determinado puerto (por ejemplo, en el puerto de origen tcp 2222):

```
sudo iptables -A INPUT -p tcp –sport 2222
```

Para evitar ataques DDoS, limita el número de peticiones por tiempo:

```
sudo iptables -A INPUT -p tcp -dport 80 -m limit 25/minute -limit-burst 100 -j ACCEPT
```

Para permitir conexiones entrantes filtrando por estado de conexión, se lanza la siguiente orden:

```
sudo iptables -A INPUT -i <interfaz> -p <protocolo> -dport <puerto> -m state -state NEW,ESTABLISHED -j ACCEPT
```

Para permitir conexiones salientes filtrando por estado de conexión, se ejecuta el siguiente comando:

```
sudo iptables -A OUTPUT -o <interfaz> -p <protocolo> -sport <puerto> -m state -state ESTABLISHED -j ACCEPT
```

Para cerrar todo el tráfico:

```
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP
```

5.3. TRAFFIC CONTROL

El firewall se puede utilizar para filtrar el tráfico y denegar peticiones no deseadas. El control de tráfico es diferente. Tiene las siguientes tareas:

- **Organización:** Cuando se organiza el tráfico, su tasa de transmisión está bajo control. La organización puede ser más que reducir el ancho de banda, también se utiliza para

suavizar picos de tráfico para mejorar el comportamiento de la red, etc. Ocurre en las peticiones de salida.

- **Programación:** Al programar la transmisión de paquetes, se mejora la interactividad para el tráfico que lo necesite al mismo tiempo que se garantiza el ancho de banda para transmisiones masivas. Se le llama también priorización y ocurre en las operaciones de salida.
- **Vigilancia:** Se encarga de gestionar el tráfico de entrada.
- **Descarte:** El tráfico que exceda un ancho de banda establecido, puede descartarse inmediatamente, tanto a la entrada como a la salida.

5.3.1. QDISC

QDISC son las iniciales de "queueing discipline". Cuando el kernel necesita enviar un paquete a una interfaz, se encola en el QDISC configurado para esa interfaz. Después, el kernel intenta recuperar tantos paquetes como sea posible del QDISC, para dárselos al driver del adaptador de red.

5.3.2. Clases

Algunos QDISCs pueden contener clases, que contienen más QDISCs. El tráfico puede ser encolado en cualquiera de los discos internos, los cuales están en las clases. Cuando el kernel intenta desencolar un paquete de un QDISC con clases, puede venir de cualquiera de las clases. Un QDISC puede, por ejemplo, priorizar ciertos tipos de tráfico frente a otros simplemente desencolando paquetes de unas clases antes que otras.

5.3.3. Filtros

Un filtro es utilizado por un QDISC con clases para determinar en qué clase se encolará un paquete. Cuando un paquete llega a una clase con subclases, es necesario elegir la clase de destino del mismo. Se pueden utilizar varios métodos para ello, uno de ellos son los filtros. El procedimiento consiste en llamar a todos los filtros que pertenecen a la clase hasta que uno de ellos devuelve un veredicto positivo. Si ninguno de ellos devuelve un veredicto positivo, se ofrece la posibilidad de utilizar otro criterio.

5.3.4. QDISC sin clases

Los QDISCs sin clases tienen las siguientes características:

- **FIFO:** Los discos más simples, de comportamiento FIFO (First In First Out) puro. Limitados en paquetes o en bytes.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **PFIFO_FAST:** QDISC estándar para kernels con "Advanced Router" activado. Consiste en una cola de tres bandas que se basa en los flags de tipo de servicio, así como la prioridad que pueda tener asignada cada paquete.
- **RED:** Son las siglas de Random Early Detection. Simula congestión física descartando paquetes aleatoriamente cuando se acerca el límite de ancho de banda configurado. Es adecuado para aplicaciones con un gran ancho de banda.
- **SFQ:** Son las siglas de Stochastic Fairness Queueing. Reordena el tráfico encolado de manera que cada sesión pueda enviar un paquete por turno.
- **TBF:** Son las siglas de Token Bucket Filter. Se utiliza para reducir el tráfico hasta un ratio configurado de forma que la compartición del ancho de banda sea seguida. Escala bien para anchos de banda grandes.

5.3.5. QDISC con clases

Los QDISCs con clases tienen las siguientes características:

- **CBQ:** (Class Based Queueing) implementa una jerarquía de compartición de links de clases. Contiene elementos de modelado y capacidades de priorización. El modelado se realiza calculando el tiempo ocioso del enlace, según el tamaño medio del paquete y el ancho de banda del enlace subyacente.
- **HTB:** (Hierarchy Token Bucket) implementa una jerarquía de compartición de links de clases conformando prácticas existentes. HTB facilita garantizar ancho de banda a las clases, a la vez que permite la especificación de los límites superiores para el intercambio entre clases. Contiene elementos de modelado basados en TBF y puede priorizar clases.
- **PRIO:** Es un contenedor sin modelado para un número configurable de clases que se desencolan en orden. Así, es posible priorizar el tráfico, donde las clases menos prioritarias pueden enviar si las más prioritarias no tienen paquetes disponibles. Los bits de tipo de servicio se respetan por defecto.

5.3.6. tc

tc es el software que se utiliza para configurar el control de tráfico en el kernel de Linux. Los comandos disponibles son los siguientes:

- **add:** Añade un QDISC, clase o filtro a un nodo.
- **remove:** Se puede eliminar un QDISC especificando su manejador. Todas las subclases y sus QDISCs hijos se eliminan automáticamente, así como cualquier filtro adjunto a ellos.
- **change:** Algunas entidades pueden ser modificadas "in-situ". Comparte sintaxis con "add", con la excepción de que el manejador no puede cambiarse, ni tampoco el padre. En otras palabras, no se puede mover un nodo.
- **replace:** Realiza un remove/add casi atómico sobre un id de nodo existente. Si aún no existe, es creado.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **link:** Solo está disponible para QDISCs y realiza un reemplazo donde el nodo ya debería de existir.

5.4. INET/XINET

Para otorgar más seguridad a nivel de red, existe inet, un demonio presente en muchos sistemas Unix que se encarga de orquestar servicios de Internet dentro de una determinada instancia. Sus siglas vienen de InterNET service Daemon. Para cada servicio configurado, escucha peticiones de clientes que se conectan. Las peticiones se gestionan levantando el proceso apropiado o redirigiéndolas a algún proceso existente, aunque algunos servicios simples (como echo) son prestados directamente por inetd.

Debido a limitaciones de seguridad del diseño original de inetd, se ha reemplazado por **xinetd**.

La configuración de xinetd se encuentra en el archivo /etc/xinetd.conf y las configuraciones de los servicios que soporta se encuentra en el directorio /etc/xinetd.d.

El siguiente bloque es un ejemplo de configuración para un servicio:

```
service time
{
        disable        = yes
        type           = INTERNAL
        id             = time-stream
        socket_type    = stream
        protocol       = tcp
        user           = root
        wait           = no
}
```

Como se mencionaba anteriormente, xinetd se puede configurar cambiando los parámetros del archivo /etc/xinetd.conf. Las diferentes variables que se pueden encontrar en este fichero se enumeran a continuación:

- **disable:** Con el valor "yes" no ejecutará el servicio.
- **type:** Hay tres tipos de servicios: INTERNAL (si el servicio lo presta xinetd)), RPC (cuando se basa en una llamada de procedimiento remota) y UNLISTED (cuando el servicio no está ni en /etc/services ni en /etc/rpc).
- **id:** Es el identificador único del servicio.

- **socket_type:** Determina cómo se va a realizar la transmisión a través del servicio. Puede ser "stream", "dgram" o "raw".
- **user:** Es el usuario propietario de la ejecución del servicio.
- **wait:** Con el valor "yes" xinetd no recibirá peticiones para el servicio si tiene una conexión.

Existen más opciones que están disponibles en el manual (man xinetd.conf).

5.5. COMPONENTES DE RED E INTERFACES

Una interfaz de red física es un componente de hardware que conecta una computadora a una red informática y posibilita la existencia de una red de computadoras. Para controlarla, Linux dispone de varias herramientas que explotan su funcionalidad.

ifconfig es un programa disponible en varias versiones del sistema operativo UNIX. Permite desplegar y configurar varios parámetros de las interfaces de red residentes en el núcleo.

Si se ejecuta sin parámetros, ifconfig devolverá información sobre todas las interfaces de red que estén funcionando.

5.5.1. Configurar una interfaz de red

ifconfig puede utilizarse para configurar (o reconfigurar) una interfaz de red.

Para asignar una IP estática a una interfaz, se especifica la interfaz y la dirección IP:

```
sudo ifconfig wlan0 11.123.32.1
```

Para asignar una máscara de red a una interfaz, se utiliza la keyword "netmask" y la dirección de la máscara de red:

```
sudo ifconfig eth1 netmask 255.255.255.0
```

Para configurar una dirección broadcast, utilizar la keyword "broadcast" e indicar la dirección:

```
sudo ifconfig wlan1 broadcast 172.11.12.1
```

Todos estos comandos se pueden hacer en una sola llamada:

```
sudo ifconfig eth0 192.168.2.3 netmask 255.255.255.0 broadcast 192.168.2.5
```

Para deshabilitar una interfaz, se utiliza el siguiente comando:

```
sudo ifconfig eth1 down
```

Para habilitar una interfaz, se utiliza el siguiente comando:

```
sudo ifconfig eth1 up
```

5.5.2. Crear una interfaz de red virtual

La creación de una interfaz de red virtual es una tarea sencilla. Basta con lanzar el siguiente comando:

```
ifconfig eth0:0 123.123.22.22
```

El comando anterior creará una nueva interfaz de red virtual basada en la interfaz física original eth0. La única condición importante es que la interfaz física tiene que existir para la creación de la interfaz de red virtual.

Para deshabilitar la interfaz de red virtual hay que ejecutar la siguiente orden:

```
ifconfig eth0:1 down
```

5.5.3. Tabla de enrutamiento de IPs

Las tablas de enrutamiento son utilizadas por el sistema para saber a qué componente de la red debe enviar cada paquete cuando la dirección IP de destino se encuentra a múltiples saltos de distancia.

Para poder ver y configurar la tabla de enrutamiento del kernel se utiliza el comando “route”. Si se ejecuta sin parámetros en la línea de comandos, se mostrarán las entradas de la tabla de enrutamiento.

La siguiente orden añade una nueva ruta:

```
sudo route add –net 192.56.123.1 netmask 255.255.255.0
```

Para eliminar una ruta, se utiliza el siguiente comando:

```
sudo route del –net <ip destino> <interfaz>
```

5.6. HERRAMIENTAS ÚTILES

Para finalizar este apartado, se enumerarán distintas herramientas y comandos útiles para gestión y configuración de redes.

- **Netstat:**

  Netstat (network statistics) es una herramienta de línea de comandos que

muestra un listado de las conexiones activas de una computadora, tanto entrantes como salientes.

Se puede utilizar para comprobar los sockets abiertos (conexiones activas existentes en la máquina). Para encontrar los archivos correspondientes a sockets Unix abiertos, se utiliza esta herramienta de la siguiente forma:

```
netstat –xl
```

- **Fail2ban:**

  Fail2ban es una aplicación para la prevención de determinados ataques en un sistema, penalizando o bloqueando las conexiones remotas que intentan accesos por fuerza bruta, ataques de denegación de servicio, replay, etc.

  Por ejemplo, imaginemos uno de los siguientes sistemas (como servidores SSH, FTP, etc.), el cual está recibiendo un ataque de fuerza bruta con el fin de obtener acceso utilizando contraseñas típicas. Fail2ban busca en los registros de los programas que se especifiquen en las reglas que el usuario decida permitiendo aplicar penalizaciones a las IPs origen del ataque automáticamente.

  Ante una cantidad predefinida de intentos fallidos, es posible determinar la acción a tomar sobre la IP que generó el problema, pudiendo simplemente notificar por e-mail el suceso, denegar el acceso a la IP atacante, bloquearla en un determinado puerto y habilitarla en otros, etc.

- **Comandos útiles:**

  Diagnosticar el estado de la comunicación del host con otro equipo remoto:

```
ping <ip>
```

  Seguir la pista de los paquetes que vienen desde un host, mostrando las IPs de los componentes públicos por los que ha ido saltando:

```
traceroute <host>
```

  Mostrar información de un usuario (siendo root se pueden conocer más datos):

```
finger <usuario>
```

  Buscar en la base de datos de dominios para devolver información sobre el propietario del mismo:

```
whois –h <host>
```

Analizar el tráfico que circula por la red (por ejemplo, por un puerto):

```
tcpdump port 110
```

Ver el listado de sockets TCP o UDP utilizados en la máquina:

```
ss –t
ss –u
```

Conectar a un determinado host por ssh.

```
ssh <host remoto>
```

BIBLIOGRAFÍA

- Explicación extendida sobre el sistema de directorios en Linux: https://www.howtogeek.com/117435/htg-explains-the-linux-directory-structure-explained/